

backendのAPI作成・動作確認

作成が必要な初期API用のバックエンドファイル

まずは後続のAPI動作確認に向けて、バックエンド側では以下の構成でファイルを準備しておきます。

```
/flexible-attendance-app/backend/src/main
├── java/com/appspace/backend
│   ├── config
│   │   └── SecurityConfig.java
│   ├── controller
│   │   ├── AttendanceController.java
│   │   └── UserAccountController.java
│   ├── entity
│   │   ├── AttendanceRecord.java
│   │   └── UserAccount.java
│   ├── repository
│   │   ├── AttendanceRecordRepository.java
│   │   └── UserAccountRepository.java
│   └── service
│       ├── AttendanceRecordService.java
│       └── UserAccountService.java
```

```
└─ BackendApplication.java
└─ resources
    └─ application.properties
```

上記にまとめたファイルそれぞれの記述内容は、以下のとおりとなっております。

SecurityConfig.java

```
package com.appspace.backend.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
```

```

public class SecurityConfig {
    /**
     * Beanアノテーションを一言でいうなら、Springという工場に部品（オブジェクト）の作り方を教えて、工場の管理リストに登録してもらうための印。
     * SpringBootでは「インスタンスの作成や管理を丸投げする」という手法（DI：依存性の注入）が可能であり、いちいち「new MyClass()」とする必要がない。
     * Beanアノテーションを用いることで、Spring標準の動きを上書きしたり、独自のルールに登録することが出来るのである。
     */
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable()) // テスト用なのでCSRF保護を無効化
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/users/register", "/api/users/login", "/api/attendance/**").permitAll() // /attendance/** で配下をすべて許可
                .anyRequest().authenticated()
            );
        return http.build();
    }
}

```

AttendanceController.java

```

package com.appspace.backend.controller;

import com.appspace.backend.entity.AttendanceRecord;
import com.appspace.backend.service.AttendanceRecordService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

```

```
@RestController
@RequestMapping("/api/attendance")
@RequiredArgsConstructor
@CrossOrigin(origins = "https://*.app.github.dev")
public class AttendanceController {

    private final AttendanceRecordService attendanceService;

    /**
     * 出勤打刻を受け付ける
     * POST http://localhost:8080/api/attendance/clock-in?accountId=ユーザーID
     */
    @PostMapping("/clock-in")
    public ResponseEntity<AttendanceRecord> clockIn(@RequestParam String accountId) {
        try {
            AttendanceRecord record = attendanceService.clockIn(accountId);
            return ResponseEntity.ok(record);
        } catch (RuntimeException e) {
            // 既に二重出勤などのエラーが発生した場合
            return ResponseEntity.badRequest().body(null);
        }
    }
}
```

```
/**
 * 退勤打刻を受け付ける
 * POST http://localhost:8080/api/attendance/clock-out?accountId=ユーザーID
 */
@PostMapping("/clock-out")
public ResponseEntity<AttendanceRecord> clockOut(@RequestParam String accountId) {
    try {
        AttendanceRecord record = attendanceService.clockOut(accountId);
        return ResponseEntity.ok(record);
    } catch (RuntimeException e) {
        // 出勤データがない、または既に退勤済みの場合
        return ResponseEntity.badRequest().body(null);
    }
}

/**
 * 打刻履歴の取得API の実装のため追加
 * 打刻履歴一覧を取得する
 * GET http://localhost:8080/api/attendance/history?accountId=ユーザーID
 */
@GetMapping("/history")
public ResponseEntity<java.util.List<AttendanceRecord>> getHistory(@RequestParam String accountId) {
    java.util.List<AttendanceRecord> history = attendanceService.getAllRecords(accountId);
    return ResponseEntity.ok(history);
}
```

```
/**
 * ステータス確認API の実装のため追加
 * 現在のステータスを確認する
 * GET http://localhost:8080/api/attendance/status?accountId=ユーザーID
 */
@GetMapping("/status")
public ResponseEntity<String> getStatus(@RequestParam String accountId) {
    String status = attendanceService.getCurrentStatus(accountId);
    // 文字列をそのまま返すとJSONとして扱いにくいいため、シンプルなテキストで返します
    return ResponseEntity.ok(status);
}
}
```

UserAccountController.java

```
package com.appspace.backend.controller;

import com.appspace.backend.entity.UserAccount;
import com.appspace.backend.service.UserAccountService;
```

```

import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController // 「画面 (HTML)」ではなく「データ (JSON)」を返す窓口であることを示す (htmlの場合は@Controller)
@RequestMapping("/api/users") // コントローラーが扱うURLの共通ルート
@RequiredArgsConstructor
@CrossOrigin(origins = "https://*.app.github.dev") // Codespacesのフロントエンドからのアクセスを許可
/**
 * GitHub Codespacesでは、フロントエンド (Vue.js) とバックエンド (Spring Boot) でURLが異なる
 * @CrossOriginの設定がないとセキュリティ制限で通信がブロックされることになる
 */
public class UserAccountController {

    private final UserAccountService userService;

    /**
     * ユーザー新規登録を受け付けるエンドポイント
     * POST http://localhost:8080/api/users/register
     */
    @PostMapping("/register")
    public ResponseEntity<UserAccount> register(@RequestBody UserAccount account) {
        // @RequestBodyで、届いたJSONデータ (ユーザー名やパスワードなど) を、自動的に UserAccount オブジェクトに変換して取り込む
        try {
            UserAccount savedAccount = userService.registerUser(account);
            return ResponseEntity.ok(savedAccount);
        } catch (RuntimeException e) {
            // 重複エラーなどが起きた場合は、400 Bad Request を返す
            return ResponseEntity.badRequest().body(null);
        }
    }
}

```



```
/**
 * ログイン処理を受け付けるエンドポイント
 * POST http://localhost:8080/api/users/login
 * SecurityConfig.javaに、このエンドポイントを許可する設定を追加している
 */
@PostMapping("/login")
public ResponseEntity<UserAccount> login(@RequestBody UserAccount loginRequest) {
    return userService.authenticate(loginRequest.getUserId(), loginRequest.getPassword())
        .map(user -> {
            // 最終ログイン日時を更新するなどの処理をここに書くことも可能
            return ResponseEntity.ok(user);
        })
        .orElse(ResponseEntity.status(401).build()); // 認証失敗時は 401 Unauthorized
}
```

AttendanceRecord.java

```
package com.appspace.backend.entity;

import jakarta.persistence.*;
```

```
import lombok.Data;
import java.time.LocalDateTime;

@Entity
@Table(name = "apply_attendance_records")
@Data
public class AttendanceRecord {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String accountId; // UserAccountのaccountIdと紐付け

    private LocalDateTime clockIn; // 出勤時刻
    private LocalDateTime clockOut; // 退勤時刻

    @Column(length = 20)
    private String status; // 例: "出勤中", "退勤済み"

    private String memo;
}
```

UserAccount.java

```
package com.appspace.backend.entity;

import jakarta.persistence.*;
import lombok.Data;
import java.time.LocalDateTime;
import java.util.UUID;

@Entity
@Table(name = "apply_user_accounts")

@Data // GetterやSetterを自分で書く必要がなくなる
public class UserAccount {

    @Id
    @Column(length = 36)
    private String accountId; // UUIDを利用した16桁～36桁の値[cite: 1]

    @Column(nullable = false, length = 30)
    private String userName;

    @Column(nullable = false, unique = true)
    private String userId; // メールアドレス[cite: 1]
```

```
@Column(nullable = false)
private String password; // BCryptハッシュ化済みパスワード[cite: 1]
```

```
@Column(nullable = false)
private int isAuth = 0; // 0:一般, 1:管理者[cite: 1]
```

```
@Column(nullable = false)
private int quitDemand = 0; // 退会申請フラグ[cite: 1]
```

```
@Column(columnDefinition = "TEXT")
private String about;
```

```
private LocalDateTime createdAt;
private LocalDateTime lastLoginAt;
```

```
// レコード作成前に自動でUUIDを生成・日時をセットする
```

```
@PrePersist // データベースに保存される直前に、Java側で自動的にUUIDと作成日時を生成する
```

```
protected void onCreate() {
    if (this.accountId == null) {
        this.accountId = UUID.randomUUID().toString();
    }
    this.createdAt = LocalDateTime.now();
}
```

```
}
```

AttendanceRecordRepository.java

```
package com.appspace.backend.repository;

import com.appspace.backend.entity.AttendanceRecord;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.Optional;

@Repository
public interface AttendanceRecordRepository extends JpaRepository<AttendanceRecord, Long> {

    // 特定のユーザーの、最新の打刻記録を1件だけ取得する
    // これにより「前回出勤したのか、退勤したのか」を判定できます
    Optional<AttendanceRecord> findFirstByAccountIdOrderByIdDesc(String accountId);

    // 打刻履歴の取得API の実装のため追加
    java.util.List<AttendanceRecord> findByAccountIdOrderByClockInDesc(String accountId);
}
```

UserAccountRepository.java

```
package com.appspace.backend.repository;

import com.appspace.backend.entity.UserAccount;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.Optional;

@Repository
public interface UserAccountRepository extends JpaRepository<UserAccount, String> {
    // JpaRepository<UserAccount, String>
    // これを継承するだけで、save()（保存）、findAll()（全件取得）、deleteById()（削除）といった標準機能が自動的に使えるようになる

    // 仕様書に基づき、ログインID（メールアドレス）でユーザーを検索するメソッドを追加[cite: 1]
    // Optionalなのは、ユーザーが見つからなかった場合に「NULL」ではなく「空の状態」として安全に扱えるようするため
    Optional<UserAccount> findById(String userId);

    // ユーザー名が含まれているものを検索する（管理者用のアカウント検索などで利用）[cite: 1]
    java.util.List<UserAccount> findByNameContaining(String userName);
}
```

AttendanceRecordService.java

```
package com.appspace.backend.service;
```

```
import com.appspace.backend.entity.AttendanceRecord;
import com.appspace.backend.repository.AttendanceRecordRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.time.LocalDateTime;

@Service
@RequiredArgsConstructor
@Transactional
public class AttendanceRecordService {

    private final AttendanceRecordRepository repository;

    /**
     * 出勤打刻
     */
    public AttendanceRecord clockIn(String accountId) {
        // 最新のレコードを確認し、既に出勤中（退勤していない）ならエラーにする
        repository.findFirstByAccountIdOrderByIdDesc(accountId)
            .ifPresent(lastRecord -> {
                if (lastRecord.getClockOut() == null) {
                    throw new RuntimeException("既に提出済みの出勤記録があります。退勤を先に完了してください。");
                }
            });
    }
}
```

```
        AttendanceRecord record = new AttendanceRecord();
        record.setAccountId(accountId);
        record.setClockIn(LocalDateTime.now());
        record.setStatus("出勤中");
        return repository.save(record);
    }

    /**
     * 退勤打刻
     */
    public AttendanceRecord clockOut(String accountId) {
        // 最新のレコードを探し、出勤データがない、または既に退勤済みならエラーにする
        AttendanceRecord lastRecord = repository.findFirstByAccountIdOrderByIdDesc(accountId)
            .orElseThrow(() -> new RuntimeException("出勤記録が見つかりません。"));

        if (lastRecord.getClockOut() != null) {
            throw new RuntimeException("既に退勤済みです。");
        }

        // 既存のレコードに退勤時刻を書き込む
        lastRecord.setClockOut(LocalDateTime.now());
        lastRecord.setStatus("退勤済み");
        return repository.save(lastRecord);
    }
}
```



```
/**
 * 打刻履歴の取得API の実装のため追加
 * 特定のユーザーの全打刻履歴を取得する
 */
public java.util.List<AttendanceRecord> getAllRecords(String accountId) {
    return repository.findByAccountIdOrderByClockInDesc(accountId);
}

/**
 * ステータス確認API の実装のため追加
 * 現在の打刻ステータスを取得する
 * @return "CLOCKED_IN" (出勤中), "CLOCKED_OUT" (退勤済み/未出勤)
 */
public String getCurrentStatus(String accountId) {
    return repository.findFirstByAccountIdOrderByIdDesc(accountId)
        .map(record -> {
            if (record.getClockOut() == null) {
                return "CLOCKED_IN";
            } else {
                return "CLOCKED_OUT";
            }
        })
        .orElse("CLOCKED_OUT"); // 記録が一つもない場合
}
}
```

UserAccountService.java

```
package com.appspace.backend.service;

import com.appspace.backend.entity.UserAccount;
import com.appspace.backend.repository.UserAccountRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.Optional;

@Service
@RequiredArgsConstructor // これをつけることで、repository を自動的に繋いでくれる（依存性の注入）コードが生成される
@Transactional // データベースの処理中にエラーが起きたら、処理を自動でキャンセル（ロールバック）してデータが壊れるのを防ぐ
public class UserAccountService {

    private final UserAccountRepository repository;
    // Spring Securityが提供する強力なハッシュ化ツール
    private final BCryptPasswordEncoder passwordEncoder = new BCryptPasswordEncoder();

    /**
     * 新規ユーザー登録（パスワードをハッシュ化して保存）
     */
    public UserAccount registerUser(UserAccount account) {
        // 1. メールアドレス（userId）の重複チェック
        if (repository.findById(account.getUserId()).isPresent()) {
            throw new RuntimeException("このメールアドレスは既に登録されています。");
        }
    }
}
```

```

    // 2. パスワードをBCryptでハッシュ化してセット
    String encodedPassword = passwordEncoder.encode(account.getPassword());
    account.setPassword(encodedPassword);

    // 3. データベースへ保存
    return repository.save(account);
}

/**
 * ログインID（メールアドレス）でユーザーを探す
 */
public Optional<UserAccount> findById(String userId) {
    return repository.findById(userId);
}

/**
 * ユーザー認証（ログインチェック）
 * ハッシュ化されたパスワード（$2a$10$...）と、ユーザーが入力した生のパスワード（mysecretpassword）を安全に照合するために設けている
 * このサービス呼び出すための「ログイン窓口（エンドポイント）」は、UserController.javaに設けられている
 * @param userId ログインID（メールアドレス）
 * @param rawPassword 入力された生のパスワード
 * @return 認証成功時はユーザー情報、失敗時は空
 */
public Optional<UserAccount> authenticate(String userId, String rawPassword) {
    return repository.findById(userId)
        .filter(user -> passwordEncoder.matches(rawPassword, user.getPassword()));
}
}

```

BackendApplication.java

```
package com.appspace.backend;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BackendApplication {

    public static void main(String[] args) {
        SpringApplication.run(BackendApplication.class, args);
    }

}
```

curlコマンドによる登録テスト

フロントエンドがない状態でも、バックエンドの「窓口（API）」が正しく機能しているか確認するのは、非常に重要なステップです。

GitHub Codespacesの**新しいターミナル**（Spring Bootを動かしているのとは別のタブ）を開いて、以下の手順でテスト用データを送ってみましょう。

以下のコマンドをコピーして、ターミナルに貼り付けて実行してください。

```
curl -X POST -H "Content-Type: application/json" \
-d '{
  "userId": "test-user@example.com",
  "userName": "テストユーザー",
  "password": "mysecretpassword",
  "about": "これはテスト登録です。"
}' \
http://localhost:8080/api/users/register
```

コマンドの解説

- `-X POST` : データを送信する（POSTメソッド）ことを指定。
- `-H "Content-Type: application/json"` : 送るデータをJSON形式に指定。
- `-d '{...}'` : 実際に送るデータの中身。
- `http://localhost:8080/api/users/register` : `UserAccountController` で作成した受付窓口の住所。

成功かどうかの判定

実行後、ターミナルに以下のようなレスポンスが返ってくれば成功です！

```
{
  "accountId": "a1b2c3d4-e5f6-...", // UUIDが自動生成されている
  "userName": "テストユーザー",
  "userId": "test-user@example.com",
  "password": "$2a$10$...",          // パスワードがハッシュ化（暗号化）されている！
  "isAuth": 0,
  "quitDemand": 0,
  "about": "これはテスト登録です。",
  "createdAt": "2026-05-07T..."    // 日時が自動セットされている
}
```

チェックポイント

1. password: 入力した `mysecretpassword` そのものではなく、`$2a$10$...` という長い文字列に化けていますか？これが **BCryptによるハッシュ化** が成功している証拠です。
2. accountId: 何も指定していませんが、ランダムな英数字（UUID）が入っていますか？これが **Entityでの自動生成** が機能している証拠です。

エラーが出た場合

- 400 Bad Request: 同じメールアドレスですでに登録されている可能性があります（Serviceで書いた重複チェックが効いています）。
- Connection refused: Spring Bootが未起動か、ポート番号が誤っています。

まずはこのコマンドを一度実行してみて、どのような結果が返ってくるかを確認してみてください。

ターミナル上で何のレスポンスも返ってこなかった場合

何も表示されずに終了したというのは、一見不安になりますが、実は「通信自体は届いたが、レスポンスの受け取り方で何かが起きた」ということを示しています。

通常、`curl` でエラーが起きれば「Connection refused」などが出ます。

何も出ないのは、サーバー側が処理をしたものの、ターミナル側で表示しきれていないか、処理が静かに終わってしまった可能性があります。

より詳細な状況を把握するために、**「詳細ログ付きのcurlコマンド」**を試してみましょう。

詳細ログ付きで再送する

以下のコマンドをコピーして実行してください。末尾に `-v`（verbose：詳細）を追加しています。

```
curl -v -X POST -H "Content-Type: application/json" \
-d '{
  "userId": "test-user@example.com",
  "userName": "テストユーザー",
  "password": "mysecretpassword",
  "about": "これはテスト登録です。"
}' \
http://localhost:8080/api/users/register
```

これで何がわかるか？

- 画面に `>` や `<` で始まる大量の文字が出ます。
 - `< HTTP/1.1 200 OK` と出ていれば、**実は登録に成功しています**。
 - `< HTTP/1.1 404` や `500` の場合、プログラム側のどこかに不備があります。

Spring Boot側の「ログ」を確認する

`curl` を打った瞬間、Spring Bootを起動している方のターミナルに何か変化はありませんでしたか？

- **白い文字（INFOログ）**が新しく流れていれば、通信は届いています。
- **赤い文字（ERRORログ）**が出ていれば、データベースへの保存などで失敗しています。

考えられる「よくある原因」

もし上記でも何も出ない場合、以下の2点が怪しいです。

1. **ポートの不一致**: Spring Bootが実は `8080` 以外で立ち上がっていないか？（起動ログに `Tomcat initialized with port(s): 8080 (http)` と出ているか確認）。
2. **パッケージ名の不整合**: 各Javaファイル（Entity, Repository, Service, Controller）の一番上にある `package ...;` の記述が、実際のフォルダ構成と1文字でもズレていると、Springが「これはコントローラーだ！」と認識してくれず、無視（スルー）されてしまいます。

まずは `-v` 付きのコマンドを実行してみて、`< HTTP/1.1 ...` の後続く数字が何番になっているかを確認してみましょう。

401エラーの場合の対処法

curlコマンドを送信したことで返ってきたレスポンスに `HTTP/1.1 401` という数字が記載されていた場合、これは「Unauthorized（認証されていません）」ということを意味します。

Spring Bootのプロジェクト作成時に「Spring Security」というライブラリを入れたため、標準設定では**すべての通信にログイン（ユーザー名とパスワード）が必要なロックがかかった状態**になっています。

これが原因でこの401エラーが発生してしまったのです。

ターミナルでのレスポンスでは、`X-Frame-Options: DENY` なども表示されているはずなのですが、これはセキュリティのための壁がしっかり立っている証拠です。今はテストのために、この壁を一時的に開ける必要があります。

`application.properties` に以前「セキュリティを無効化する」設定を1行書きましたが、ライブラリのバージョンによってはそれだけでは不十分な場合があります。

最も確実なのは、「**セキュリティ設定用のクラス**」を作成して、`/api/users/register` へのアクセスを**全許可**する設定を書くことです。

1. 新しいクラスの作成

`backend/src/main/java/com/example/demo` の中に、新しく `config` というフォルダを作り、その中に `SecurityConfig.java` というファイルを作成してください。

2. コードの記述

以下の内容をコピー＆ペーストしてください。

```
package com.example.demo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;
```

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable()) // テスト用なのでCSRF保護を無効化
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/users/register").permitAll() // 登録窓口だけは誰でもOKにする
                .anyRequest().authenticated() // それ以外はロック
            );
        return http.build();
    }
}
```

3. 再起動と再テスト

1. `SecurityConfig.java` を保存します。
2. Spring Bootを `Ctrl + C` で一度止めてから、再度 `./mvnw spring-boot:run` で起動してください。
3. 起動後、先ほどの `curl -v ...` のコマンドをもう一度実行してみてください。

次の結果はどうなる？

今度は `< HTTP/1.1 200 OK` が返ってくるはずです！ `< HTTP/1.1 200` というステータスコードに加え、最後に返ってきた JSON データの内容に注目してください。

ここが「成功」の証拠

- `"accountId": "04ed4539-..."` : Java側で指定しなかったIDが、UUIDとして自動生成されている。
- `"password": "$2a$10$..."` : `mysecretpassword` が、解読不能なハッシュ値に変換されている。
- `"createdAt": "2026-05-07T..."` : サーバーの時間で正しく登録日時が記録される。

これで、「**フロントエンド** → **コントローラー** → **サービス** → **リポジトリ** → **データベース**」というバックエンドの全行程が1本の線で繋がったのです。

データベースの中身を自分の目で見てみよう

現在はメモリ上のデータベース（H2 Database）を使用しているため、ブラウザからその中身を直接のぞき見ることができます。

1. Spring Boot を動かしたまま、ブラウザで以下のURLを開きます。

`https://（あなたのCodespacesのURL）/h2-console`

（ポート8080の地球儀マークで開いたURLの末尾を `/h2-console` に書き換える）

2. ログイン画面が出たら、以下を確認して「**Connect**」を押します。

○ **JDBC URL:** `jdbc:h2:mem:testdb`

○ **User Name:** `sa`

○ **Password:** （空欄）

3. 左側のツリーに `APPLY_USER_ACCOUNTS` というテーブルがあるはずです。

4. SQLコマンド `SELECT * FROM APPLY_USER_ACCOUNTS;` を入力して「Run」を押してみてください。

先ほど `curl` で送ったデータが1行表示されれば、データの保存も完璧です！